Object-Centric Grounding for End-to-End Driving

Master's thesis progress and research stay proposal

Richard Cernansky

Faculty of Informatics and Information Technologies
Slovak University of Technology in Bratislava

Supervisor: Matej Halinkovic, PhD.

Interview, Mobile Perception Systems Lab, TU Eindhoven

26 October 2026

Agenda

1. **Master's thesis**
   - Research problem and objectives
   - Technical approach and methodology
   - Progress, preliminary results and how I read them
2. **Research stay proposal**
   - Motivation and the research gap
   - Hypotheses, methodology, expected contributions
3. **Connection between the two**
   - What the thesis delivers that the proposal needs
   - Scientific questions for the stay, fit with the lab

Outline

1 Master's Thesis

2 Research Stay Proposal

3 Connection

Trajectory prediction

Modular stacks detect, then track, then predict[1, 2].

- The interface between stages is just a list of boxes and identities
- Whatever information not encoded in a box is discarded
- *Errors propagate forward* and cannot be corrected downstream

End-to-end perception and prediction removes that interface.
ViP3D[3] carries persistent agent queries through time, and the trajectory decoder reads the query features.

Problem with ViP3D's Agent Queries

- learned embedding of fixed size
- identical in every frame, scene
- reference points from that embedding, so Query X starts in the same place on an empty motorway and at a busy intersection

Objectives

Goal of the thesis

A deep learning architecture for end-to-end perception and prediction in which the agent queries originate in the point cloud rather than in a learned embedding.

3 research questions:

1. What does the query-based formulation achieve once the queries are **derived from LiDAR**?
2. Once the queries already carry geometric evidence, what **does the camera still contribute**, and to which metrics and classes?
3. A query that starts near its object **needs less layers to correct**. What does that buy in decoder depth and in inference latency?

Query initialisation is interesting

Input-dependent reference points are known to be better and to reduce the decoder depth needed for convergence.

Our reading of the detection literature:

- **DETR**[4] queries are learned vectors with no explicit position at all.
- **Deformable DETR**[5] makes the position explicit as a reference point, predicted from the query embeddin - **input-independent**
- Its two-stage variant takes reference points from the centres of the top-scoring encoder proposals instead, and outperforms the single-stage one.
- **TransFusion**[6] do the initialisation in BEV, heatmap peaks into object queries.

Nobody has applied this to end-to-end perception and prediction, where the query must also survive across frames.

So how is the query generated?

A class-wise heatmap is predicted over the BEV grid with CenterPoint-style[7] Gaussian targets. For each of the top-K peaks:

- the reference point is the metric position of the peak,
- the content is the BEV feature sampled at that cell plus an embedding of its class.

What makes this work with tracking

- Only the slots left free by the tracker are filled
- Agents already being followed keep the queries they carry (suppression)
- initialisation concerns newly appearing objects alone.

Local-max NMS is 3×3 and excludes pedestrian and bicycle classes

Methodology

Training, two stages following the TransFusion recipe[6]

- Stage 1: LiDAR model alone, 20 epochs
- Stage 2: stage 1 frozen, camera layer on top, 6 epochs (so fusion cannot destabilise)
- No data augmentation in any run (future work)

Data, nuScenes[8]

- 700 training and 150 validation scenes, keyframes at 2 Hz
- Persistent instance identities, three-frame clips, 6 s futures
- Seven classes: car, truck, bus, trailer, motorcycle, bicycle, pedestrian

Metrics

- EPA[3] is the primary criterion
- minADE, minFDE and MR average only over matched agents, so detecting fewer objects improves them
- EPA normalises by the annotated agents and subtracts false positives
- Detection and tracking metrics reported alongside

Protocol

- Identical evaluation code, class set and thresholds for every row
- Best epoch selected by EPA on validation
- Latency over 200 timed iterations

InitVip3D Architecture

- Full LiDAR branch, heatmap head and query initialisation implemented on top of ViP3D.
- Two-layer TransFusion decoder, deformable BEV attention in layer 0 and SMCA camera attention in layer 1, with a geometric fallback for queries that project into no camera.
- Two-stage schedule, with stage 2 on a frozen stage-1 model, plus image-guided initialisation as the ablation that separates *fusion* from *guide*.
- Camera-visibility masking, so configurations with fewer than six cameras are not supervised on objects they cannot see.
- Evaluation harness for detection, tracking and prediction metrics, plus latency and parameter benchmarking and score analysis.
- Multiple training runs across batch size, query count and LiDAR sweep count.

Cost: stage 1 is $\approx$ 38 min per epoch on one node of 8$\times$H200, $\approx$ 13 h end to end. A full validation pass is $\approx$ 100 min per checkpoint.

Preliminary results

nuScenes validation, all seven classes, All rows at effective batch 8.

Run	EPA ↑	minADE ↓	minFDE ↓	MR ↓	Latency (ms) ↓
ViP3D *as published*[3], 24 ep	0.245	1.47	2.21	0.237	64
Stage 1, LiDAR only, 300 q	0.293	0.956	1.776	0.183	90
Stage 1, LiDAR only, 100 q	0.256	1.022	1.937	0.239	81
Stage 1, LiDAR only, 5 sweeps					58
Stage 2, + SMCA camera	**0.318**	**0.882**	**1.590**	**0.176**	130

Reading the results

1. **Query origin matters more than the fusion does.** heatmap-derived queries take minADE from 1.47 to 0.956 and EPA from 0.245 to 0.293 (at the same parameter count and against a baseline trained in the same harness).
2. **Camera fusion brings small benefit with increased latency.** Adding the camera layer on top of the frozen LiDAR model moves EPA 0.293 to 0.318. The camera still contributes once the queries already carry geometry, which is the answer to question 2.
3. **Reduction in number of sweeps is favorable.**
4. **PointPillar LiDAR backbone is weak on detections.** This detection inability propagates to the prediction causing errors. Trying more complex LiDAR backbone is the future step.

Outline

1 Master's Thesis

2 Research Stay Proposal

3 Connection

Motivation

Vision-language-action models are a route to interpretable end-to-end driving. **Alpamayo-R1**[9] generates a Chain-of-Causation reasoning trace, and a flow-matching action expert decodes a trajectory conditioned on it.

The reported gains concentrate exactly where imitation learning is weakest: 12 % improvement in planning accuracy on challenging cases and a 35 % reduction in close-encounter rate in closed-loop simulation.

So the trace is not decorative. It is a functional intermediate representation between perception and planning.

The structural observation this proposal starts from

End-to-end driving removed the hand-crafted interface between perception and planning. AR1 restores an interface, but expresses it in natural language, where nothing can check it.

The research gap

A trace says *"the black SUV from the left is going straight"*[9]. Nothing in the architecture requires that SUV to exist.

- The image tokens the reasoning conditions on are shaped only by the language-modelling objective. There is no detection head, no depth or segmentation supervision, and no object-level entry anywhere in the KV-cache that the reasoning and the action expert read.
- Weak visual grounding is listed among the limitations of supervised fine-tuning, and the mitigation is post-hoc, a reasoning model grading traces during RL. Nothing constrains the reasoning at inference time.
- The second gap is temporal. Causal locality in the labelling protocol and a KV-cache rebuilt each frame mean the scene is re-derived every step and no identity persists.

Gap

Driving VLAs couple perception to reasoning through an unverifiable, memoryless interface, grounded in neither sensor evidence nor time.

Objectives and hypotheses

Introduce an explicit, sensor-grounded, temporally propagated object representation into the VLA sequence, and evaluate whether it improves the fidelity of the reasoning and the quality of the resulting trajectories.

H1 **Grounding failure is measurable and non-negligible.** Traces from the released AR1 model refer to object categories absent from the scene at a rate significantly above that of the human-verified annotations on the same clips.

H2 **An object-centric span reduces it.** Conditioning the reasoning on a span of detection-derived agent queries lowers that rate without degrading open-loop accuracy or closed-loop safety.

H3 **Temporal propagation extends the benefit beyond the observation window.** Propagating queries across frames beats re-detecting them independently, specifically under occlusion and reappearance.

All work fine-tunes released checkpoints with the public Alpamayo recipes and the AlpaSim simulator. Nothing here needs training from scratch.

Phase 1: measure the failure before fixing it

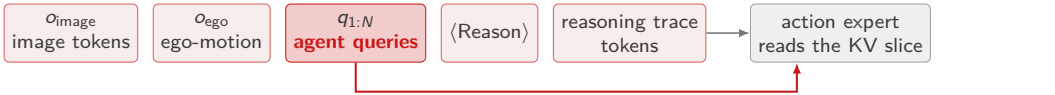
No training required. Run the released checkpoint over clips that also carry 3D annotations.

1. Parse each generated trace for referenced categories and coarse positions.
2. Score a reference as unsupported when no annotated object of that category exists in that region at that time.
3. Apply the identical measurement to the human-written traces, which calibrates how often the parser is simply wrong rather than the model.

Why this phase comes first

It is cheap, it needs no new annotation, and it resolves whether the premise of the whole proposal holds. A negative result is still a contribution, the first quantitative characterisation of object-level hallucination in a released driving VLA, and it redirects the remaining months rather than wasting them.

Phase 2: the object span



Agent queries are projected into the backbone embedding space and inserted before the reasoning tokens. Causal attention then exposes every query to every reasoning token, so grounding reaches the language path and the control path at once.

The detection loss becomes the auxiliary objective AR1 asks for

Their future work[9] suggests depth or scene-flow auxiliaries to improve the visual backbone. Supervising the span with a detection loss is a more targeted version of that idea, aimed at grounding the reasoning rather than at improving features in general.

Controls, because the span might simply be ignored

The dominant failure mode of an added input is that a model with an existing path to the answer learns to ignore the redundant one. Two ablations are built in from the start, not added afterwards.

Span masking Replace the queries with learned null embeddings at inference and measure the change in the generated traces. No change means the span is inert.

Span corruption Substitute queries taken from a different scene. A trace that adopts the foreign objects shows the span is genuinely read.

Attention mass placed on the query positions by the reasoning tokens is logged throughout as a continuous diagnostic, so the question is answered by a curve rather than by a single ablation number.

N is held in the tens and selected by decision relevance rather than raw confidence, and the token cost is reported alongside latency.

Phase 3: temporal propagation, and the grounding reward

Queries are propagated between frames with ego-motion compensation rather than re-detected independently. Evaluation targets occlusion and reappearance, where the stateless variant cannot by construction retain an agent that has left the observation window.

If time permits, the RL stage gains a grounding reward analogous to AR1's reasoning-action consistency reward. Parse the trace into referenced categories, compare against the query set, score the match. Rule-based, so it needs neither new annotation nor a judge model.

Example

Trace: *"yielding to the pedestrian on the right, lead vehicle stopped"*
Referenced: {pedestrian, vehicle} Query set: {car, car, truck}
Score: pedestrian unmatched, penalty applied.

This item is explicitly optional and is the first thing I drop if the schedule slips.

Evaluation protocol and expected contributions

Evaluation.

- *Grounding*: unsupported-reference rate per category and scenario, plus attention mass on the span.
- *Open-loop*: minADE$_6$ and minFDE over 6.4 s, nominal and long-tail splits separately, since the claimed benefit concentrates in the latter.
- *Closed-loop*: close-encounter rate, off-road rate and AlpaSim score, with an occlusion breakdown.
- *Cost*: added tokens, added parameters, end-to-end latency against the reported 99 ms budget.

Expected contributions.

1. The first quantitative characterisation of object-level hallucination in a released driving VLA, with a reusable protocol. This stands regardless of H2 and H3.
2. An architecture and training recipe for a sensor-grounded, temporally propagated object span, with ablations that establish whether it is read rather than ignored.
3. A characterisation of the token-budget against grounding trade-off, directly relevant to embedded deployment.
4. A workshop or conference submission, with code released against the public checkpoints.

Outline

1 Master's Thesis

2 Research Stay Proposal

3 Connection

How the stay builds on the thesis

The thesis produces exactly the representation AR1 lacks.

The thesis delivers

- A compact set of agent queries, tens rather than thousands.
- Grounded in sensor data by a detection loss.
- Carrying persistent identity across frames through a memory bank.
- Initialised from the data rather than from a constant.

The stay requires

- A span short enough to insert into a token sequence.
- A supervision signal that grounds it in the scene.
- State that survives beyond the observation window.
- Queries that reflect this scene, not a prior over scenes.

In one sentence

The thesis answers how to build the representation. The stay asks what a reasoning model does with it.

Scientific questions to be addressed during the stay

1. How often does a released driving VLA refer to objects that are not there, measured against human-written traces on the same clips?
2. Does conditioning the reasoning on an explicit, detection-supervised object span reduce that rate, and can the span be shown to be used rather than ignored?
3. Does propagating the queries across frames extend the benefit to occlusion and reappearance, which causal locality and a per-frame KV-cache cannot reach?
4. What does the span cost in tokens, parameters and latency, and is that affordable on automotive-grade embedded hardware?
5. Can a rule-based grounding reward replace a judge model in the RL stage?

Fit with the Mobile Perception Systems Lab

- The lab's long-term goal is spatio-temporal reasoning that lets a mobile platform anticipate future events, identified as a bottleneck in crowded inner-city settings. **H3 is a concrete instance of that problem.**
- The test vehicle carries a 360-degree camera setup with LiDAR and radar, which enables the multimodal variant where LiDAR-derived queries ground an otherwise camera-only backbone. **My preliminary work is LiDAR-based and transfers directly.**
- Work in the lab on token-efficient vision transformers addresses precisely the cost an added span incurs, which the Phase 2 token-budget analysis measures.
- Radar perception suggests extending the multimodal variant to the conditions where a camera backbone is least reliable.
- Closed-loop evaluation would run in the TU/e digital twin alongside the public AlpaSim scenario sets.

Three things to take away

1. **A result.** Deriving agent queries from a LiDAR heatmap instead of a learned embedding more than doubles EPA and roughly halves minADE against a baseline I trained in the same harness, at an unchanged parameter count. Camera fusion adds to that rather than replacing it.
2. **A limitation I can explain.** Tracking lags because the classification score is well ranked but badly calibrated against a hard survival gate, and I can point to the measurement that shows it.
3. **A proposal that follows from both.** AR1 reasons about objects it has no grounded representation of. The thesis builds that representation. The stay measures whether supplying it makes the reasoning true and the driving safer.

Thank you

Technical discussion and questions

Backup: why EPA is the primary metric

EPA as defined by ViP3D[3]:

$$\mathrm{EPA} = \max\left(0,\ \frac{N_{\mathrm{hit}} - \alpha N_{FP}}{N_{\mathrm{GT}}}\right), \qquad \alpha = 0.5$$

- Predicted agents are matched to annotated agents by Hungarian assignment on centre distance, gated at 2 m, in the current frame.
- A matched agent counts as a hit only if its minFDE_K stays within 2 m at the horizon.
- minADE, minFDE and MR average only over matched agents, so a model that detects fewer objects can improve all three.
- N_{GT} counts the annotated agents that survive the whole horizon, and false positives are subtracted, so EPA penalises both failure modes at once.

Backup: the tracking interface

The model's inference behaviour is controlled by parameters that no gradient touches, because they sit behind hard comparisons and a top-K selection.

Parameter	Value
confirm a new track	0.40
keep a track alive	0.35
miss tolerance	5 frames
heatmap NMS kernel	3×3
active-track suppression	5×5 cells
queries K	300

True positives score 0.73 ± 0.18 and false positives 0.57 ± 0.16. The ranking is good, the absolute calibration is not. With that spread a confirmed track dips below the 0.35 gate in roughly 17 % of frames, is deleted, and is reacquired under a new identity.

This costs tracking metrics while leaving detection metrics largely intact, which is why AMOTA and EPA disagree.

Any gain found by tuning these is free at inference. No retraining, no extra parameters.

Backup: batch size and query count sweep

Stage 1, LiDAR only, nuScenes validation. Learning rate was sqrt-scaled with the batch size.

Stage-1 variant	minADE ↓	EPA ↑	Note
300 queries, effective batch 8	0.956	0.293	best, the row in the results table
300 queries, batch 16	0.997	0.277	
300 queries, batch 32	1.114	0.236	
300 queries, batch 64	1.061	0.256	
100 queries, batch 8	1.022	0.256	
100 queries, batch 16	1.528	0.234	

Accuracy degrades close to monotonically with batch size even with the learning rate scaled, which is consistent with the usual small-batch-noise argument. I report it as an observation rather than an explanation, because the flat-minima claim is testable on the checkpoints I already have and I have not tested it yet.

Backup: configuration details

LiDAR branch

- Point cloud range ± 51.2 m, pillars of 0.2 m, 20 points per pillar.
- PillarFeatureNet $5 \to 64$, PointPillarsScatter to 512×512.
- SECOND with strides $[2, 2]$, SECONDFPN to $384 \times 256 \times 256$, so 0.4 m per BEV cell.
- 10 accumulated sweeps, 5 features per point.

Heatmap head

- CenterPoint nuScenes task grouping, so pretrained per-task weights still line up.
- Gaussian focal loss, radius from the box footprint.

Decoder

- Embedding 256, 8 heads, FFN 512, dropout 0.1.
- Layer 0 deformable BEV attention, 4 sampling points per query.
- Layer 1 SMCA over 4 FPN levels and 6 cameras, anisotropic σ from the projected box size.
- Reference points and sizes updated after each layer, sizes detached after layer 0.

Schedule

- Stage 1: 20 epochs, camera path frozen.
- Stage 2: 6 epochs, LiDAR path frozen, lr $3.5\mathrm{e}{-4}$.

Backup: my questions for the lab

1. Which AR1 checkpoints and which parts of the Alpamayo training stack are available inside the lab, and is AlpaSim usable for closed-loop evaluation in February?
2. Do the campus recordings carry 3D annotations, or would Phase 1 run on the public clips and the campus data enter only in Phase 2 and 3?
3. What compute would a fine-tuning run have, and how is it scheduled?
4. Is the LiDAR and radar on the test vehicle time-synchronised and calibrated against the 360-degree camera rig to the accuracy that query-level projection needs?
5. Who else in the group works on the reasoning side, and would this sit alongside an existing effort or beside it?
6. What is a realistic publication target for a four-month result in your experience?

References I

[1] L. Chen et al. "End-to-end autonomous driving: Challenges and frontiers". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 46.12 (2024), pp. 10164–10183.

[2] M. Liang et al. "PnPNet: End-to-End Perception and Prediction with Tracking in the Loop". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2020, pp. 11553–11562.

[3] J. Gu et al. "ViP3D: End-to-End Visual Trajectory Prediction via 3D Agent Queries". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2023, pp. 5496–5506.

[4] N. Carion et al. "End-to-End Object Detection with Transformers". In: *European Conference on Computer Vision (ECCV)*. 2020, pp. 213–229.

[5] X. Zhu et al. "Deformable DETR: Deformable Transformers for End-to-End Object Detection". In: *International Conference on Learning Representations (ICLR)*. 2021.

[6] X. Bai et al. "TransFusion: Robust LiDAR-Camera Fusion for 3D Object Detection with Transformers". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2022.

[7] T. Yin et al. "Center-Based 3D Object Detection and Tracking". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2021.

References II

[8] H. Caesar et al. "nuScenes: A Multimodal Dataset for Autonomous Driving". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2020.

[9] NVIDIA AV Applied Research Team. *Alpamayo-R1: Bridging Reasoning and Action Prediction for Generalizable Autonomous Driving in the Long Tail*. 2025. arXiv: 2511.00088 [cs.CV].